

This project was about creating and implementing a Vigenère cipher and using it to encrypt and decrypt messages. I created a brute-force password cracker that uses a provided dictionary to crack the encrypted message. After testing the cracker, I optimized the code to improve its efficiency with longer key lengths.

Here are the decrypted messages:

1. "MSOKKJCOSXOEKDTOSLGFWCMCHSUSGX" key length = 2; firstWordLength = 6
Key = KS, decrypted message: "CAESARSWIFEMUSTBEABOVESUSPICION"
2. "PSPDYLOAFSGFREQKKPOERNIYVSDZSUOVGXSRIPWERDIPCFSDIQZIASEJVC
GXAYBGYXFPSREKFMEXEBIYDGFKREOWGXEQSXSXKXGYRRRVMEKFFIPIWJSKF
DJMBGCC" keyLength=3; firstWordLength = 7
Key = KEY, decrypted message:
"FORTUNEWHICHHASAGREATDEALOFPOWERINOTHERMATTERSBUTESPECIALL
YINWARCANBRINGABOUTGREATCHANGESINASITUATIONTHROUGHVERYSLIGHT
FORCES"
3. "MTZHZEQKASVBDOWNMWMKMNYIIHVWPEXJA" keyLength=4; firstWordLength = 10
Key = IWKD, decrypted message: EXPERIENCEISTHETEACHEROFALLTHINGS
4. "SQLIMXEEKSXMDOSBITOTYVECRDXSCRURZYP0HRG" keyLength=5;
firstWordLength = 11
Key = KELCE, decrypted message:
"IMAGINATIONISMOREIMPORTANTTHANKNOWLEDGE"
5. "LDWMEKPOPSWNOAVBIDHIPCEWAETYRVOAUPSINOVDIEDHCDSELHCCPVHRP
OHZUSERSFS" keyLength=6; firstWordLength = 9
Key = HACKER, decrypted message:
EDUCATIONISWHATREMAINSATERONEHASFORGOTTENWHATONEHASLEARNE
DINSCHOOL

Here are the cracking times for the above ciphers:

Task, Key Length	Before Optimization	After Optimization
1, 2	0.0021385 s	0.0590362 s
2, 3	0.0271595 s	0.103849 s
3, 4	0.69411 s	0.284871 s
4, 5	18.5593 s	4.52427 s
5, 6	454.72 s	110.272 s

I optimized the cracker by checking the decrypted letters against valid dictionary prefixes, allowing the program to reject impossible words earlier. I also changed how the program generates keys so that it increments the letters directly instead of repeatedly using division and modulo. The cracking time overall increases as the key length increases. The first two tests got slower, but that is because the optimized code has to spend some time creating the dictionary prefix set before it begins cracking. For the larger key lengths, this extra setup time becomes negligible compared to the amount of time saved by rejecting the impossible words early. Unfortunately, I ran out of time to test and compare Task 6 times, but the larger tests show that my optimized cracker was about 4 times faster than my original brute-force cracker. Based on this pattern, I would expect Task 6 to also be cracked roughly 4 times faster using the optimized version.